

Why the BOM Generation Pipeline Returns Zero

Leather / lining / sub-material lines: a junior-developer walkthrough of every bottleneck found, what was fixed, and what a genuinely correct result looks like — verified against real files, real code, and a real database.

Scope of this report

This document explains, end to end, why generating a BOM (Bill of Materials) for a leather garment style kept coming back with **qty_per_garment = 0**, **dcm_source = "provisional"** for the main_material / sub_material / lining lines — across two different real datasets tested in this session. Every claim below was checked by actually running the project's own code against real files (a real DXF pattern, a real order sheet PDF, a real spreadsheet, and a real Postgres database) — nothing here is guessed or assumed.

If you are new to this codebase: read §1 first (the mental model of how a BOM gets built), then §2 (the bugs, in the order they block each other), then §3 (the proof it actually works once fed correctly).

1. How a BOM is supposed to get built (read this first)

A BOM line for a leather/fabric material needs two independent things to both succeed:

- **A line to exist at all** — something has to decide “this style has a main_material line called X”. This comes **only** from the SPEC SHEET (a garment tech-pack: materials, POM measurements, accessories, lining). The order sheet is never consulted for this.
- **A real quantity for that line** — how many dm² of that material one garment needs. This comes from the DXF cutting pattern (best), or a previously-confirmed template, or a POM-based estimate, or — if none of those resolve — a deliberate placeholder of zero (dcm_source="provisional"), flagged for a human to fill in at the cutting table.

The pipeline, in the order the code actually runs it (app/modules/bom/service.py, method generate_bom, and the DCM ladder _resolve_dcm):

```
1. Extract the SPEC sheet (Gemini/Groq) -> ExtractedSpec
2. Flatten it into a plain dict (_spec_attributes) -> spec_sheet.attributes
3. Build BOM line "seeds" from that dict ONLY -> _build_line_seeds(attrs)
   attrs["leather_quality"] -> ONE main_material seed
   attrs["sub_materials"] -> sub_material seeds
   attrs["lining"] -> ONE lining seed
   attrs["accessories"] -> accessory seeds
4. For every DCM-eligible seed, resolve a real qty -> _resolve_dcm(...)
   Source 1 : an already-confirmed template for this exact style
   Source 1b : a template via a resolved pattern reference
   Source 1c : the confirmed DXF pattern's own area data <-- what we fixed
   Source 2 : a similar style's confirmed DCM
   Source 3 : POM-area heuristic (needs real POM measurements)
   else : PROVISIONAL (qty 0, confidence 0, flagged)
```

The single most important fact in this whole report: **step 3 never looks at the order sheet**. The order sheet's own “material” column (e.g. “GOAT SUEDE”, “BURNED SUEDE”) is captured into the database (OrderStyle.material) but is **never read again** by anything that builds a BOM line. If there is no real spec sheet, most BOM lines simply don't exist — they don't even reach step 4 to be zeroed out.

2. Every bug found this session, in the order they blocked each other

These are listed in the order you would actually hit them running the real pipeline end to end. Each one was individually confirmed (a real crash, a real wrong value, or a real code-read), fixed, and re-verified.

2.1 Celery workers crashed with “Event loop is closed”

Where: `app/core/database.py`, `app/modules/bom/tasks.py`

Every Celery task ran on a brand-new `asyncio` event loop (`asyncio.run` per task), but the database engine used a connection *pool* built for one long-lived FastAPI event loop. A connection opened on task 1's loop got reused by task 2 on a different (already-dead) loop → crash the moment SQLAlchemy tried to “ping” it before reuse.

Fix: Gave Celery its own engine using **NullPool** (a fresh DB connection per checkout, never reused across loop boundaries).

2.2 Worker process crashed with a foreign-key resolution error

Where: `app/core/celery.py`

The FastAPI process imports every model module at startup so SQLAlchemy knows every table exists (main.py's own “Model imports” block). The Celery worker is a *separate process* that never runs main.py — it only auto-discovered each module's `tasks.py`, not its `models.py`. The first real ORM write in the worker tried to resolve a foreign key to a table (client) that had simply never been registered in that process.

Fix: Import every model module in `celery.py` too, mirroring main.py exactly.

2.3 A confirmed pattern attachment pointed at the wrong table entirely

Where: `app/modules/bom/models.py` + Alembic migration

`OrderStyle.pattern_reference_id`'s foreign key pointed at `pattern_reference` — an unrelated, legacy “follow pattern code X” table that nothing writes to. The real DXF-parsed data lives in a differently-named table, `pattern_extraction`. Confirming an uploaded DXF onto a style crashed with a foreign-key violation, because the id only ever exists in the second table.

Fix: Repointed the FK to `pattern_extraction.id` via a new Alembic migration, applied to the live DB.

2.4 The DXF auto-match queried the wrong table and a non-existent field

Where: `app/modules/bom/repository.py` (`patterns_for_client`)

This function — meant to list a client's uploaded DXF patterns for auto-suggesting a match — queried the same wrong legacy table as 2.3, and its caller read a `.style_signature` attribute that table doesn't even have. This would have raised `AttributeError` the instant any pattern existed for a client.

Fix: Pointed it at `PatternExtraction` (the real table) and restricted to `is_current`.

2.5 A confirmed DXF attachment was silently ignored by BOM generation

Where: `app/modules/bom/service.py (generate_bom)`

Even after fixing 2.3/2.4, `generate_bom` never looked at the pattern a human had explicitly confirmed. It always independently re-matched a pattern by (style signature, client id) from scratch — and that auto-match silently misses whenever the DXF's own signature differs from the style's (a shared base pattern uploaded under a different code) or the uploader's client id doesn't match the order's (see 2.7). The human's confirmation had zero effect on the generated BOM.

Fix: Added a `confirmed_pattern_id` parameter, used *only* when the attachment's status is literally “confirmed” (not merely auto-suggested) — a direct id lookup instead of a fuzzy re-match.

2.6 A duplicate DXF re-upload crashed instead of being a no-op

Where: `app/modules/bom/repository.py, service.py`

The database correctly rejects two DXF rows with the same (style, file-hash) as a duplicate — but the code never checked for that case first, so re-uploading the identical file crashed with an unhandled database error instead of returning the existing row.

Fix: Catch the duplicate-key error, look up and return the existing row — matching the method's own documented “idempotent” contract.

2.7 A staff user's DXF upload could never carry a real client id

Where: `app/modules/bom/router.py (upload_pattern)` — fixed this session, confirmed

The upload endpoint always used `getattr(user, "client_id", None)` — the *uploading staff member's own* client id, which for a Direct Manager / MD account is always `None` (staff aren't scoped to one client). So an uploaded pattern's client id could never match the order's real client id, silently breaking the auto-match path in 2.5 even after everything else was fixed.

Fix: The endpoint now accepts an explicit `client_id` for staff uploaders; a CLIENT-role caller stays pinned to their own (matches the existing pattern in `clients/router.py`).

2.8 A submission with no client failed invisibly, deep in a background worker

Where: `app/modules/bom/router.py (create_order_breakdown)` — fixed this session, confirmed

A submission opened without a client id would be accepted by the “start breakdown” endpoint (202 Accepted), silently fail inside the Celery worker minutes later, and release its own claim — with no HTTP-visible error anywhere. The only trace was a worker log line.

Fix: The endpoint now checks the submission's client id *before* claiming/enqueuing, and returns an immediate 422 if it's missing.

2.9 Two smaller keyword / schema mismatches

Where: `app/modules/bom/service.py, repository.py, models.py`

(a) a POM-suggestion function was called with the wrong keyword-argument name (`llm_json` instead of the real `llm_json_call`), crashing every LLM fallback for an unresolved measurement term. (b) the code tried to save a “suggested / confirmed” status and a confidence score for an LLM-guessed POM mapping onto a table that had neither column.

Fix: Fixed the keyword name; added the two missing columns (with a migration) so a suggested mapping can be reviewed instead of crashing.

3. The real root cause for THIS dataset: there is no spec sheet

Everything in §2 was fixed and verified against a dataset that *does* have a real spec sheet (a matched order+spec+DXF triple for style CR1-02F5-JK26). But the user's own three files were tested directly, and this is where the real bottleneck for THAT dataset lives.

3.1 The three files, as they actually are

File	What it actually is (verified by opening it)
jackiee-order-sheet.pdf	A real, handwritten multi-page ORDER SHEET for client PAKKAR — 11 pages, many styles (CLERMONT, ...)
johnpeter.xlsx	A digitized spreadsheet with columns Date / Style / Colour / Article / size-grid / TOTAL QTY — the
CLERMONT_15-06-26-P53.dxf	A real CAD cutting pattern, 53 pieces, one graded size ('50'). This is a genuine PATTERN file.

None of the three is a spec sheet (a garment tech-pack listing lining, interlining, accessories, and POM measurements). The system was never given one for this style.

3.2 What actually happens when the real code is run on johnpeter.xlsx

The real spec-extraction function (extract_spec, a genuine call to Gemini — not mocked) was run directly against johnpeter.xlsx. Its real output:

```
style_no: 'CARNABY' <- just the FIRST row of the sheet; CLERMONT's
own rows are silently never looked at
article: 'GOAT SUEDE'
materials: {'leather_quality': 'GOAT SUEDE', 'leather_substance_mm': []}
sub_materials: [] <- EMPTY
lining: {} <- EMPTY
interlining: {} <- EMPTY
accessories: [] <- EMPTY
measurements (POMs): 0 rows <- EMPTY
extracted_by: 'gemini' (a real, successful LLM call)
```

The extraction itself worked — Gemini even cleverly inferred a leather type from the “Article” column. But an order-breakdown spreadsheet genuinely does not contain lining, interlining, accessory, or measurement information, so those fields come back empty because that data was never there to begin with. This is not a bug in the extractor — it is the correct behaviour when handed the wrong kind of document.

3.3 What that means for the BOM's line-seed builder

Feeding that exact output into the real _build_line_seeds function (service.py:420-479) produces:

Line	Would it be created?	Why
main_material (leather)	YES — one line, “GOAT SUEDE”	attrs.get("leather_quality") was truthy (lucky — Gemini inferred it)
sub_material	NO	attrs.get("sub_materials") was an empty list
lining	NO	attrs.get("lining") was never set — the extraction's lining dict was entirely empty
interlining	NO	same reason
accessory	NO	attrs.get("accessories") was an empty list

So even before DCM resolution is considered, four of the five material/accessory line types would never appear in the BOM at all for this exact input — not zero, *absent*. Only main_material survives, and only by luck.

4. Proof: the DXF math itself is correct, when fed properly

To separate “the DXF/DCM engine is broken” from “the pipeline never reaches the DXF engine for this dataset”, the real DXF was parsed and the real DCM functions were called directly, using genuine data pulled from the live database (seeded species-yield table) — no shortcuts, no mock values.

4.1 What the real DXF parser actually returned

```
master_size: None <- no graded sizes; this is a SINGLE-size export
fabric_matrix: {'50': {'LEATHER': 13.31, 'VELLUTO': 2.95, 'LINING': 12.78,
'TEXTILE': 6.6, 'NYLON': 5.95, '{Non Definito}': 0.43}}
(units: net square feet, per piece-group, at the one graded size)
```

A missing master_size can be a bug elsewhere in the code (see §5.2) — but the size-resolver has a specific, deliberate escape hatch for exactly this case: “if there is only one size in the file, always use it, whatever size was asked for” (its own code comment literally names this file as the reference example: pattern.py:48, # single-size export (CLERMONT '50')). So this file hits zero size-matching problems.

4.2 Fabric-label recognition — also correct

Each raw CAD label was run through the real lexicon lookup (attribute_fabrics, fabric_roles.py). All five real labels resolved correctly (only the literal placeholder “{Non Definito}” — Italian for “undefined” — was left unmapped, correctly):

CAD label	Resolved role	Category	Is leather?
LEATHER	main	main_material	YES
VELLUTO (IT: suede)	sub_material	sub_material	YES
LINING	lining	lining	no
TEXTILE	lining	lining	no
NYLON	lining	lining	no
{Non Definito}	— unmapped —	—	—

4.3 The actual computed numbers (real function calls, real DB yield data)

Species classification: dcm.species_of(“GOAT SUEDE”) → “goat”. Yield table pulled live from the seeded database: {sheep: 2.50, goat: 2.10, calf: 2.30, lamb: 2.50, default: 2.30}. Feeding all of this into the real dcm_for_category() function:

Category	Net pattern area (sq ft)	Yield factor	x 9.290304 (sq ft->dm²)	Real qty_per_ga
main_material (LEATHER)	13.31	2.10 (goat)	9.290304	259.67 dm²
sub_material (VELLUTO)	2.95	2.10 (goat)	9.290304	57.55 dm²
lining (LINING+TEXTILE+NYLON)	25.33	1.15 (fabric waste)	9.290304	270.62 dm²

These are real, sane numbers — 259.67 dm² (≈ 2.6 m²) of goat suede for one full leather jacket body is entirely plausible once hide waste is included. **This proves the DXF-to-DCM computation chain itself has no bug** for this file — the size resolver, the fabric lexicon, the species classifier and the yield-lookup all produced correct, non-zero results the moment they were given matching, correctly-scoped inputs.

5. Ranked bottleneck list — fix in this order

#	Bottleneck	Blocks	Status
1	No real spec sheet for a style → <code>_build_line_seeds</code> Everything slows to a crawl, no material/lining generated, bugs at \$60 content/process gap. Needs a real spec sheet.	Everything slows to a crawl, no material/lining generated, bugs at \$60 content/process gap. Needs a real spec sheet.	Not fixed
2	<code>generate_bom</code> ignored a human-confirmed DXF attachment (\$2.5) → auto-match fails	Any attachment (\$2.5)	FIXED this session
3	DXF auto-match used the wrong <code>client_id</code> source (\$2.7) → staff-uploaded DXF	Ev (\$2.7)	FIXED this session
4	Wrong-table FK + wrong-table query for patterns (\$2.6/2.4) → any DXF attachment	CS (\$2.6/2.4)	FIXED this session
5	Silent <code>submission_missing_client_id</code> failure (\$2.8) → any submission opened without a client_id	Any (\$2.8)	FIXED this session
6	Celery event-loop / connection-pool crash (\$2.1) → every single background task, interrupted	Ev (\$2.1)	FIXED this session
7	Missing model imports in worker process (\$2.2) → First ORM write per worker process	First ORM write per worker process (\$2.2)	FIXED this session

6. Recommendation

For the user's exact three files: the pipeline is not “broken” in the sense of a crash or a wrong formula — it is missing an input it was designed to require (a real spec sheet per style). Two honest paths forward:

- **Get/produce a real spec sheet** for CLERMONT (materials, lining, accessories, POM grid) and upload it as the SPEC document — the DXF math already proven in §4 will then attach to real seeded lines automatically.
- **Or** extend `_build_line_seeds` to fall back to the DXF's own `fabric_roles` when the spec provides nothing for `sub_material/lining` — a real code change, not a data-collection task, and a reasonable scope for a follow-up ticket.

For the dataset that DOES have a real spec sheet (CR1-02F5-JK26): all seven numbered bugs in §2 are now fixed and verified; generating that style's BOM again (fresh, not the stale cached one) should now produce real, non-provisional numbers.